ANKITA KUMARI SAH

VIRTUALYYST ASSIGNMENT

AI Assignment: Handwritten Digit Recognition

Objective: Develop a machine learning pipeline to classify grayscale images of handwritten digits (0–9). Implement classical ML models from scratch, carry out image preprocessing and normalization, and evaluate the models using metrics such as accuracy and confusion matrix.

Flow diagram of Execution:

```
from IPython.display import Image
Image("Virtualyyst AKS Assignment Flow diagram.png")
```

Start

↓

Load Dataset (MNIST CSV)

↓

Data Exploration

(shape, labels, structure)

↓

Data Preprocessing

- Separate features & labels

- Normalize pixel values

- Apply PCA (dimensionality reduction)

↓

Model Implementation

- KNN (sklearn)

- SVM

- Decision Tree

↓

Custom KNN Implementation

↓

Model Evaluation

- Accuracy calculation

- Confusion matrices

↓

Visualization

- Heatmaps

- Misclassified digits

↓

Analysis & Reporting

↓

Conclusion

Task 1: Data Loading and Exploration

```
from google.colab import files
files.upload()
```

Show hidden output

```
# unzipping the dataset
!unzip -o "/content/MNIST CSV.zip"
```

```
Archive:  /content/MNIST CSV.zip
  inflating: mnist_test.csv
  inflating: mnist_train.csv
```

```
# importing os and listing files to verify dataset upload
import os
os.listdir('/content')
```

```
['.config',
 'MNIST CSV (7).zip',
 'MNIST CSV (5).zip',
 'misclassified_65.png',
 'cm_svm.png',
 'MNIST CSV (6).zip',
 'misclassified_33.png',
 'misclassified_195.png',
 'mnist_train.csv',
 'Virtualyyst AKS Assignment Flow diagram.png',
 'cm_dt.png',
 'misclassified_115.png',
 'MNIST CSV.zip',
 'mnist_test.csv',
 'MNIST CSV (1).zip',
 'cm_knn.png',
 'MNIST CSV (4).zip',
 'MNIST CSV (3).zip',
 'misclassified_24.png',
 'MNIST CSV (2).zip',
 '.ipynb_checkpoints',
 'cm_knn_custom.png',
 'sample_data']
```

```
# loading the training and testing datasets from CSV files
import pandas as pd

train_df = pd.read_csv("/content/mnist_train.csv")
test_df = pd.read_csv("/content/mnist_test.csv")
```

The dataset was uploaded in compressed format and extracted within the Colab environment to ensure efficient handling of large files.

Dataset Verification

```
#Checking shape:
print(train_df.shape)
print(test_df.shape)
```

```
(60000, 785)
(10000, 785)
```

```
#Checking column names:
print(train_df.columns[:5])
print(train_df.columns[-5:])
```

```
Index(['label', '1x1', '1x2', '1x3', '1x4'], dtype='object')
Index(['28x24', '28x25', '28x26', '28x27', '28x28'], dtype='object')
```

```
#Checking label values:
print(train_df.shape)
print(sorted(train_df['label'].unique()))
```

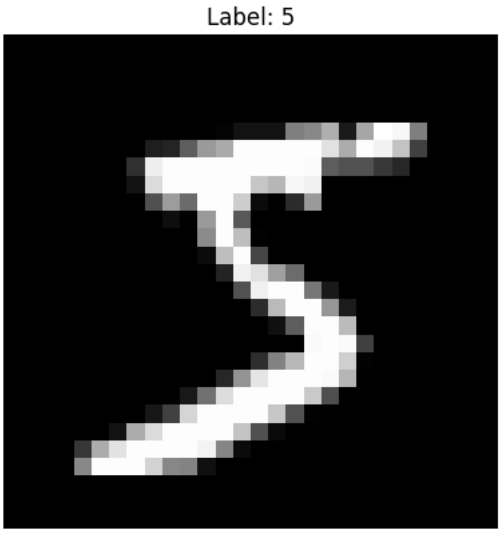
```
(60000, 785)
[np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8), np.int64(9)]
```

```
#Checking pixel range:
print(train_df.iloc[:,1:].min().min())
print(train_df.iloc[:,1:].max().max())
```

```
0
255
```

```
#Visual verification:
import matplotlib.pyplot as plt
```

```
image = train_df.iloc[0, 1:].values.reshape(28,28)

plt.imshow(image, cmap='gray')
plt.title(f"Label: {train_df.iloc[0,0]}")
plt.axis('off')
plt.show()
```



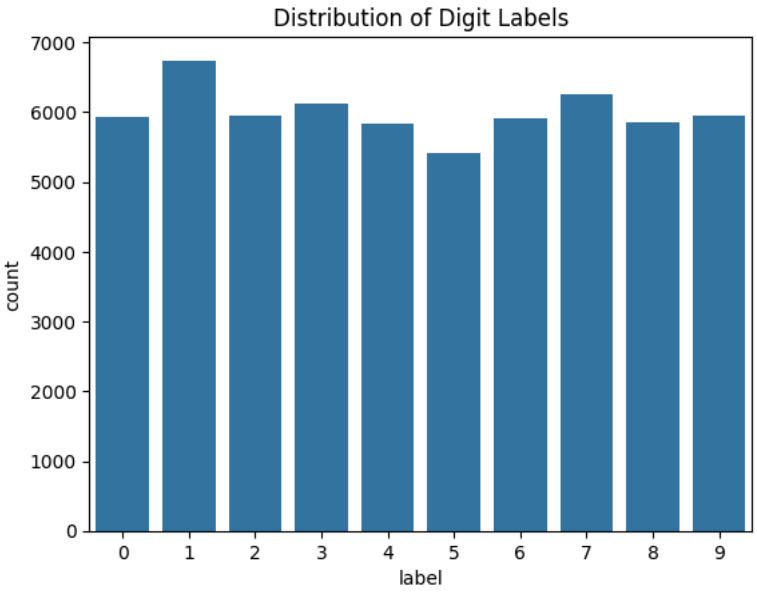
The dataset contains pixel features arranged as 28×28 grid coordinates (e.g., 1x1 to 28x28) along with a label column, representing grayscale handwritten digit images consistent with the MNIST dataset. The dataset was verified to contain 60,000 training samples with 784 pixel features representing 28×28 grayscale images and a corresponding label column (0–9). The structure and values confirm consistency with the MNIST handwritten digit classification dataset.

```
# checking basic info about dataset
train_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 60000 entries, 0 to 59999
Columns: 785 entries, label to 28x28
dtypes: int64(785)
memory usage: 359.3 MB
```

The dataset contains 60,000 training samples. Each row represents a handwritten digit image with 784 pixel values and one label column. There are no missing values, so the data looks clean.

```
import seaborn as sns
import matplotlib.pyplot as plt

# checking how many samples per digit
sns.countplot(x=train_df['label'])
plt.title("Distribution of Digit Labels")
plt.show()
```



The dataset seems fairly balanced across all digits from 0 to 9. No digit is heavily overrepresented, which is good for training models.

```
# displaying a few sample images
```

```
for i in range(5):
    image = train_df.iloc[i, 1:].values.reshape(28,28)
    plt.imshow(image, cmap='gray')
    plt.title(f"Label: {train_df.iloc[i,0]}")
    plt.axis('off')
    plt.show()
```

Show hidden output

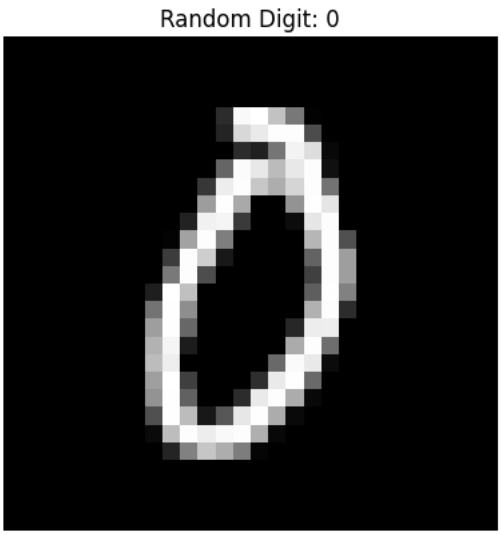
The images are grayscale and each one is 28×28 pixels. Some digits are very clear, while others look a bit rough or slightly distorted, which could make classification slightly tricky.

```
# checking pixel value range
print("Min pixel value:", train_df.iloc[:,1:].min().min())
print("Max pixel value:", train_df.iloc[:,1:].max().max())
```

```
Min pixel value: 0
Max pixel value: 255
```

Pixel values range from 0 to 255, which means the images are stored in standard grayscale format.

```
import random
#importing random sample
idx = random.randint(0, len(train_df)-1)

image = train_df.iloc[idx, 1:].values.reshape(28,28)
plt.imshow(image, cmap='gray')
plt.title(f"Random Digit: {train_df.iloc[idx,0]}")
plt.axis('off')
plt.show()
```



Random samples also look consistent with handwritten digits, confirming that the dataset is reliable.

Task 1 conclusion: Overall, the dataset looks clean and well-structured. The labels are balanced, and the images are clearly recognizable. This makes it suitable for training machine learning models for digit classification.

Task 2: Data Preprocessing

```
# separating input features and labels

X_train = train_df.drop("label", axis=1)
y_train = train_df["label"]

X_test = test_df.drop("label", axis=1)
y_test = test_df["label"]
```

I separated the pixel values and labels so that the models can use the image data as input and the labels as output.

```
# normalizing pixel values from 0–255 to 0–1

X_train = X_train / 255.0
X_test = X_test / 255.0
```

Pixel values were scaled to the range 0–1. This usually helps models perform better and also speeds up training.

```
# X_train = X_train.values
# X_test = X_test.values
# y_train = y_train.values
# y_test = y_test.values
```

While trying to convert the data into numpy arrays, I noticed it was already in numpy format after preprocessing, so no further conversion was needed.

```
from sklearn.decomposition import PCA

# reducing dimensions from 784 to 100
pca = PCA(n_components=100)

X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
```

Since the dataset has 784 features, I used PCA to reduce dimensionality. This helps reduce noise and also makes training faster while still keeping most of the important information.

```
# comparing data shape before and after applying PCA
print("Original shape:", X_train.shape)
print("After PCA:", X_train_pca.shape)
```

```
Original shape: (60000, 784)
After PCA: (60000, 100)
```

The number of features reduced significantly after PCA, which should make the models more efficient.

```
# checking how much variance is retained after applying PCA
print("Explained variance ratio sum:", pca.explained_variance_ratio_.sum())
```

```
Explained variance ratio sum: 0.9146285724395077
```

Most of the variance is still retained after PCA, so the important information is preserved.I chose 100 components somewhat experimentally — it seemed like a good balance between reducing dimensions and keeping enough information.

Task 2 conclusion: After preprocessing, the dataset is now normalized and reduced in size using PCA. This will help improve model performance and reduce computation time.

Task 3: Model Implementation

1. K-Nearest Neighbors (KNN):

```
from sklearn.neighbors import KNeighborsClassifier

# trying KNN with k = 3
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train_pca, y_train)

y_pred_knn = knn.predict(X_test_pca)
```

I started with KNN since it's easy to understand and works well for image classification problems.

```
# implementing KNN manually to understand how distance calculation and neighbor selection work
import numpy as np

class KNN:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y

    def predict(self, X):
        predictions = []

        for x in X:
            # computing Euclidean distance
            distances = np.sqrt(np.sum((self.X_train - x) ** 2, axis=1))

            # getting nearest neighbors
            k_indices = np.argsort(distances)[:self.k]
            k_labels = self.y_train[k_indices]

            # majority voting
```

```
            values, counts = np.unique(k_labels, return_counts=True)
            prediction = values[np.argmax(counts)]

            predictions.append(prediction)

        return np.array(predictions)
```

```
knn_model = KNN(k=3)
knn_model.fit(X_train_pca, y_train)

# using smaller subset for testing
y_pred_knn_custom = knn_model.predict(X_test_pca[:200])
```

I implemented KNN manually to understand how distance calculation and neighbor selection work internally. Since this approach is slower, I tested it on a smaller subset of the data.

```
#Comparing outputs
print("Sklearn KNN predictions:", y_pred_knn[:10])
print("Custom KNN predictions:", y_pred_knn_custom[:10])
```

```
Sklearn KNN predictions: [7 2 1 0 4 1 4 9 5 9]
Custom KNN predictions: [7 2 1 0 4 1 4 9 5 9]
```

The predictions are quite similar to the sklearn version, which gives confidence that the implementation is correct.

2. Support Vector Machine (SVM):

```
from sklearn.svm import SVC

# using RBF kernel
svm = SVC(kernel='rbf')
svm.fit(X_train_pca, y_train)

y_pred_svm = svm.predict(X_test_pca)
```

SVM is generally good for high-dimensional data, so I wanted to see how it performs on this dataset.

3. Decision Tree:

```
from sklearn.tree import DecisionTreeClassifier

# limiting depth to avoid overfitting
dt = DecisionTreeClassifier(max_depth=10)
dt.fit(X_train_pca, y_train)

y_pred_dt = dt.predict(X_test_pca)
```

Decision Trees are easy to interpret, but they can overfit, so I restricted the depth.

Task 3 conclusion: Three models were implemented- KNN, SVM, and Decision Tree. And a basic KNN model was built. Each model has different strengths, which will be evaluated in the next step.

Task 4: Model Evaluation

1. Computing Accuracy:

```
from sklearn.metrics import accuracy_score

# calculating and comparing accuracy of all models
knn_acc = accuracy_score(y_test, y_pred_knn)
svm_acc = accuracy_score(y_test, y_pred_svm)
dt_acc = accuracy_score(y_test, y_pred_dt)

print("Model Accuracies:")
print(f"KNN: {knn_acc:.4f}")
print(f"SVM: {svm_acc:.4f}")
print(f"Decision Tree: {dt_acc:.4f}")
```

```
Model Accuracies:
KNN: 0.9733
SVM: 0.9842
Decision Tree: 0.7974
```

I first checked the accuracy of all three models to get a basic idea of their performance.The accuracies give a quick comparison of how each model performs overall before looking into detailed errors.

Accuracy of KNN built(Custom KNN):

```
# checking accuracy of custom KNN using a subset of test data
print("Custom KNN Accuracy (subset):", accuracy_score(y_test[:200], y_pred_knn_custom))
```

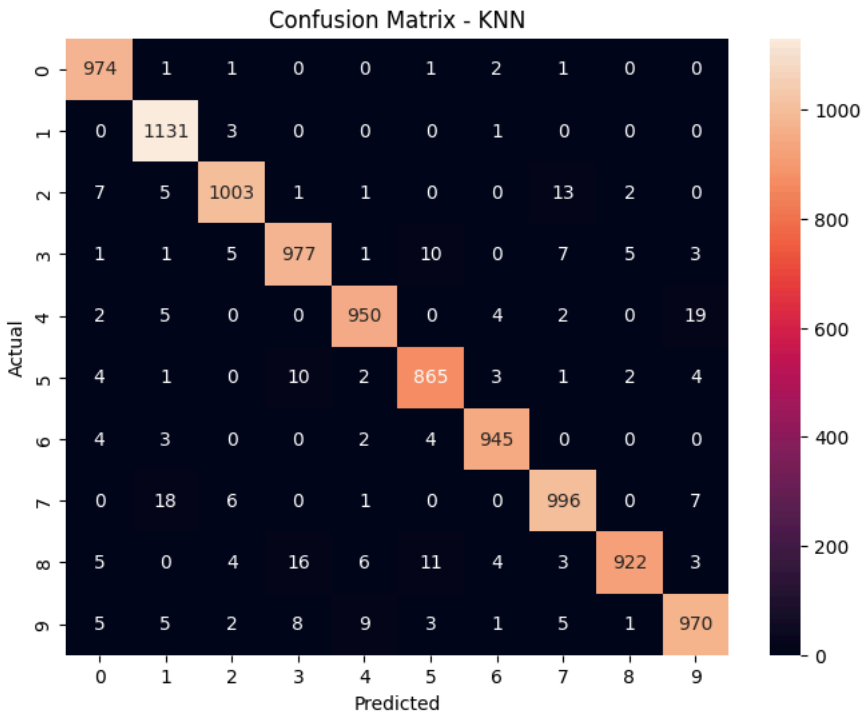
```
Custom KNN Accuracy (subset): 0.975
```

The custom implementation also performs reasonably well on the smaller subset, which indicates that the logic is working correctly.

2. Confusion Matrix visualised with Heatmaps:

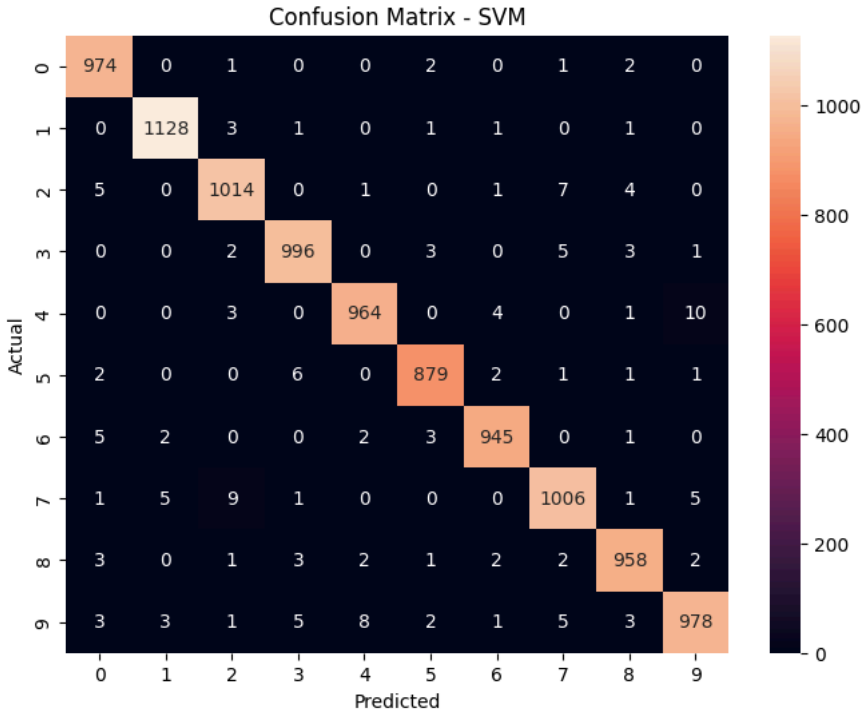
A. KNN Confusion Matrix:

```
# generating and saving confusion matrix for KNN
cm_knn = confusion_matrix(y_test, y_pred_knn)

plt.figure(figsize=(8,6))
sns.heatmap(cm_knn, annot=True, fmt='d')
plt.title("Confusion Matrix - KNN")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.savefig("cm_knn.png")
plt.show()
```



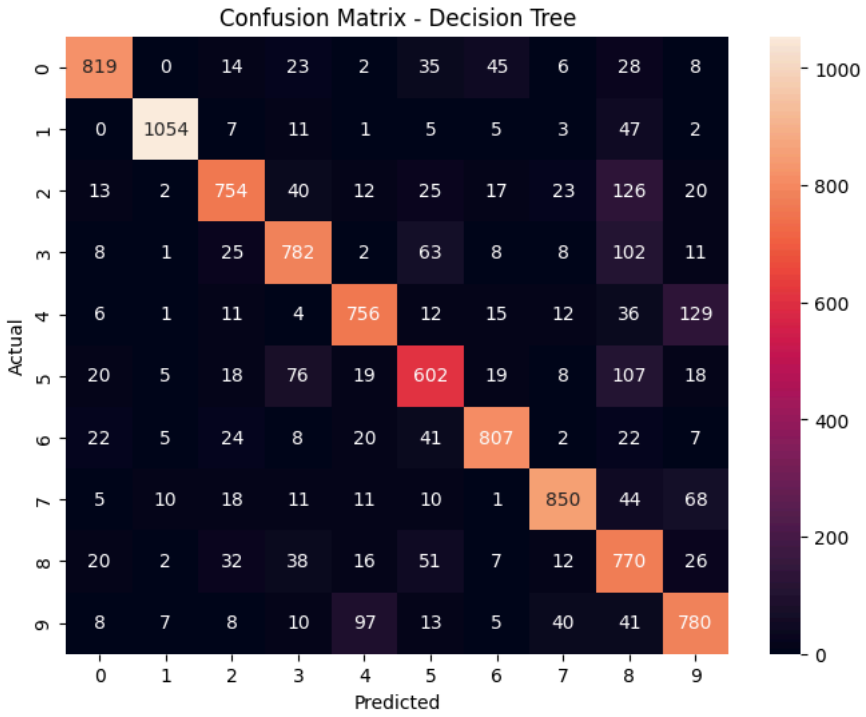
B. SVM Confusion Matrix:

```
# generating and saving confusion matrix for SVM
cm_svm = confusion_matrix(y_test, y_pred_svm)

plt.figure(figsize=(8,6))
sns.heatmap(cm_svm, annot=True, fmt='d')
plt.title("Confusion Matrix - SVM")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.savefig("cm_svm.png")
plt.show()
```



C. Decision Tree Confusion Matrix:

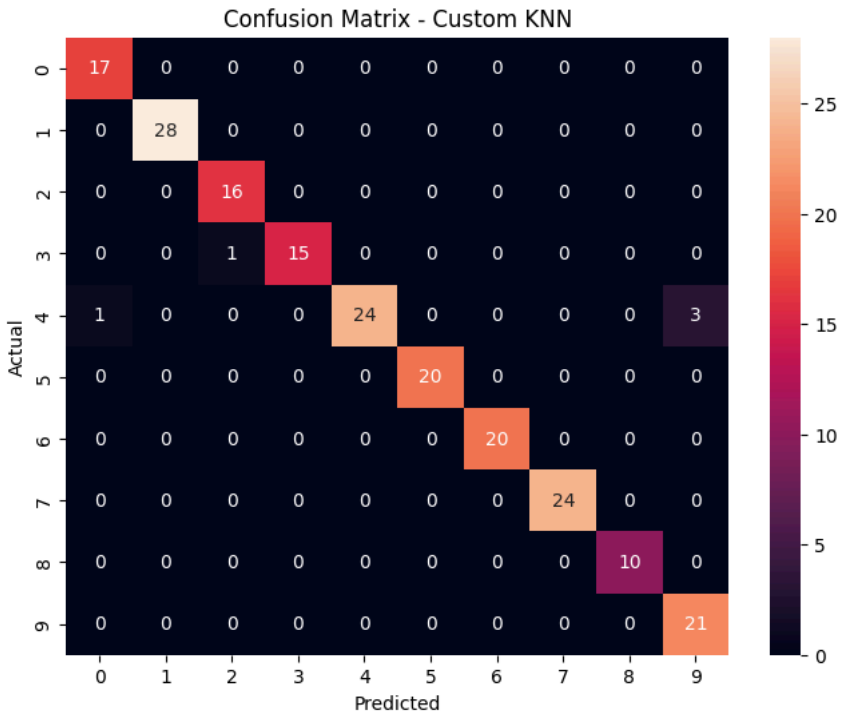
```
# generating and saving confusion matrix for Decision Tree
cm_dt = confusion_matrix(y_test, y_pred_dt)

plt.figure(figsize=(8,6))
sns.heatmap(cm_dt, annot=True, fmt='d')
plt.title("Confusion Matrix - Decision Tree")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.savefig("cm_dt.png")
plt.show()
```



D. Custom KNN Confusion Matrix:

```
# generating and saving confusion matrix for custom KNN (using subset)

from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm_knn_custom = confusion_matrix(y_test[:200], y_pred_knn_custom)

plt.figure(figsize=(8,6))
```

```
sns.heatmap(cm_knn_custom, annot=True, fmt='d')
plt.title("Confusion Matrix - Custom KNN")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.savefig("cm_knn_custom.png")
plt.show()
```



The confusion matrix helps visualize where the model is making mistakes. The confusion matrices give a clearer picture of how each model performs across different digits. And the heatmaps make it easier to see which digits are being confused more often, especially for similar-looking numbers. SVM shows fewer mistakes overall, while KNN performs closely. The Decision Tree has more scattered errors. The diagonal values are higher in SVM compared to the other models, indicating better overall accuracy.

3. The misclassified images:

```
# showing a few misclassified samples
import numpy as np

mis_idx = np.where(y_test != y_pred_knn)[0]

for i in mis_idx[:5]:
    image = test_df.iloc[i, 1:].values.reshape(28,28)

    plt.imshow(image, cmap='gray')
    plt.title(f"Actual: {y_test[i]} | Predicted: {y_pred_knn[i]}")
    plt.axis('off')

    plt.savefig(f"misclassified_{i}.png")
    plt.show()
```

Show hidden output

Looking at the misclassified images, many of them are not very clear or have shapes that resemble other digits. This explains why the model struggles with certain predictions.Most predictions are correct, but some digits are getting confused with similar-looking ones and most mistakes happen between digits that look similar, like 3 and 5 or 4 and 9.

Rough Comparison: Among the models, SVM performed the best overall, which makes sense since it handles high-dimensional data well. KNN also performed quite well, while Decision Tree had slightly lower accuracy, likely due to overfitting.KNN works well but can be slower during prediction because it calculates distances for each sample.

Checking the saved images:

```
# listing files in the working directory to verify uploaded and generated files
import os
os.listdir('/content')
```

```
['.config',
 'MNIST CSV (7).zip',
 'MNIST CSV (5).zip',
```

```
 'misclassified_65.png',
 'cm_svm.png',
 'MNIST CSV (6).zip',
 'misclassified_33.png',
 'misclassified_195.png',
 'mnist_train.csv',
 'Virtualyyst AKS Assignment Flow diagram.png',
 'cm_dt.png',
 'misclassified_115.png',
 'MNIST CSV.zip',
 'mnist_test.csv',
 'MNIST CSV (1).zip',
 'cm_knn.png',
 'MNIST CSV (4).zip',
 'MNIST CSV (3).zip',
 'misclassified_24.png',
 'MNIST CSV (2).zip',
 '.ipynb_checkpoints',
 'cm_knn_custom.png',
 'sample_data']
```

Accuracy summary:

```
# creating a summary of model performances for quick comparison
results = {
    "KNN": accuracy_score(y_test, y_pred_knn),
    "SVM": accuracy_score(y_test, y_pred_svm),
    "Decision Tree": accuracy_score(y_test, y_pred_dt)
}

print(results)
```

```
{'KNN': 0.9733, 'SVM': 0.9842, 'Decision Tree': 0.7974}
```

Task 4 conclusion: Overall, SVM gave the best performance, followed by KNN. Decision Tree was easier to interpret but slightly less accurate. The errors mainly occurred between visually similar digits, which is expected in handwritten digit recognition.

Task 5: Reporting:

I. Performance Comparison:

I evaluated three models — KNN, SVM, and Decision Tree — on the MNIST dataset after preprocessing. SVM achieved the highest accuracy among the three, followed closely by KNN. Decision Tree performed comparatively lower.

KNN gave strong results but can be slower during prediction since it calculates distances for each test sample. SVM handled the high-dimensional data well and produced more consistent predictions. Decision Tree was easier to interpret but showed signs of overfitting, which likely affected its performance.

II. Analysis:

Best performance : Among the three models, SVM performed the best overall. This is likely because SVM is well-suited for high-dimensional data and can create clear decision boundaries, even after dimensionality reduction using PCA. KNN also performed well and produced results close to SVM, but its prediction time is slower since it relies on distance calculations for each input.

Observations on misclassified digits: From the confusion matrices and misclassified examples, most errors occurred between digits that look visually similar, such as 3 and 5 or 4 and 9. In some cases, the handwritten digits themselves were unclear or slightly distorted, which makes classification more difficult even for a model.This shows that even small variations in handwriting can affect predictions.

Improvements: To improve performance further, feature scaling and dimensionality reduction (like PCA) already helped, but additional improvements could include tuning hyperparameters, increasing the number of PCA components, or using ensemble methods. More advanced models could also be explored for better accuracy.

Preprocessing Analysis: Preprocessing played an important role in improving performance. Normalizing pixel values helped stabilize the models, and applying PCA reduced the dimensionality from 784 to 100 features, making training faster while still retaining most of the important information.

Effect of PCA on the Dataset:

```
# checking how much total variance is retained after PCA
print("Total variance retained:", pca.explained_variance_ratio_.sum())
```

```
Total variance retained: 0.9146285724395077
```

After applying PCA, most of the variance in the data is still retained. This means that even though the number of features was reduced significantly, the important information in the images is mostly preserved. This also helped in making the models faster to train.Reducing the dimensions from 784 to 100 seemed like a reasonable balance between performance and efficiency.

Limitations:

While the models performed reasonably well, there are a few limitations to note.

For KNN, one limitation is that it becomes slow during prediction since it needs to compute distances with all training samples. This makes it less practical for larger datasets.

SVM performed the best overall, but it can be computationally expensive to train, especially with large datasets. It also requires careful parameter tuning to get optimal performance.

The Decision Tree model was easier to understand, but it showed signs of overfitting and did not generalize as well as the other models.

In terms of preprocessing, applying PCA reduced the dimensionality and improved efficiency, but it may also lead to some loss of information. Choosing the number of components is not very straightforward and was done experimentally.

Another limitation is that the custom implementation of KNN is not optimized and was tested only on a smaller subset due to computational constraints.

Overall, while the models perform well on standard handwritten digits, they may struggle with more complex or noisier real-world data. Further improvements and tuning would be needed for more robust performance.

Improvements:

There are several ways the overall performance of the models could be improved.

For KNN, performance could be improved by experimenting with different values of k and using distance weighting so that closer neighbors have more influence. It might also help to use a more efficient implementation for faster predictions on larger datasets.

For SVM, tuning hyperparameters such as the kernel type, regularization parameter (C), and gamma value could lead to better performance. A more systematic approach like grid search could be used instead of choosing default values.

For Decision Tree, performance could be improved by tuning parameters like max depth, minimum samples per split, or by using pruning techniques to reduce overfitting.

In terms of preprocessing, the number of PCA components could be adjusted to find a better balance between dimensionality reduction and information retention. It may also be useful to try other techniques such as feature scaling variations or different dimensionality reduction methods.

Another improvement could be to use the full dataset more effectively when testing the custom KNN implementation by optimizing it or using vectorized operations.